



भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Information Technology Guwahati

CS236 - ARTIFICIAL INTELLIGENCE LAB
LAB 0: AN INTRODUCTION TO PYTHON

INTRODUCTION TO PYTHON PROGRAMMING

1 Variables and Dynamic Typing

Python is a dynamically typed language. A variable can refer to objects of different types during execution.

```
x = 10
print(type(x))

x = 3.5
print(type(x))

x = "Python"
print(type(x))
```

2 Basic Input and Output

Input is read as a string and must be converted explicitly if required.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Name:", name)
print("Age:", age)
```

3 Conditional Statements

Python uses indentation to define blocks.

```
x = 15

if x > 10:
    print("Greater than 10")
elif x == 10:
    print("Equal to 10")
else:
    print("Less than 10")
```

4 Loops

4.1 For Loop

Iteration is commonly performed over sequences rather than indices.

```
for i in range(5):  
    print(i)
```

```
numbers = [2, 4, 6, 8]  
for num in numbers:  
    print(num)
```

4.2 While Loop

```
count = 0  
while count < 3:  
    print(count)  
    count += 1
```

5 Lists

Lists are ordered and mutable.

```
numbers = [1, 2, 3, 4]  
numbers.append(5)  
numbers[0] = 10  
print(numbers)
```

5.1 List Slicing

```
nums = [0, 1, 2, 3, 4, 5]  
print(nums[1:4])  
print(nums[:3])  
print(nums[::2])
```

6 Tuples

Tuples are ordered and immutable.

```
point = (3, 4)  
x, y = point  
print(x, y)
```

7 Dictionaries

Dictionaries store key–value pairs.

```
student = {
    "name": "Alice",
    "roll": 101,
    "marks": 85
}

print(student["name"])
student["marks"] = 90
```

8 Functions

Functions may return multiple values.

```
def compute_sum_and_product(a, b):
    return a + b, a * b

s, p = compute_sum_and_product(3, 4)
print(s, p)
```

9 Common Mistakes and Notes

- Indentation errors cause syntax errors.
- Accessing an undefined variable raises a `NameError`.
- List indices start from 0.
- Dictionary keys must be unique and immutable.
- Python does not require explicit variable declarations.

10 Assignments

1. Write a program that reads an integer and prints whether it is positive, negative, or zero.
2. Write a program that reads three integers and prints the largest among them.
3. Given a list of integers, write a program to compute the sum and average of the elements.
4. Write a program to count how many even and odd numbers are present in a given list.
5. Create a dictionary that stores student names as keys and their marks as values. Write a program to compute the average marks.
6. Write a function that takes a list of numbers and returns the minimum and maximum values.
7. Given a string, write a program to count the frequency of each character using a dictionary.
8. Write a program that generates the first n Fibonacci numbers using a loop.

OBJECT-ORIENTED AND FUNCTIONAL PROGRAMMING IN PYTHON

1 Defining Classes and Objects

Classes in Python are defined using the `class` keyword. The constructor method is named `__init__`.

```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll

    def display(self):
        print(self.name, self.roll)

s1 = Student("Alice", 101)
s1.display()
```

2 Instance Variables and Methods

Instance variables are created using the `self` reference and are local to each object.

```
class Counter:
    def __init__(self):
        self.count = 0

    def increment(self):
        self.count += 1
```

3 Operator Overloading

Python allows operators to be redefined for user-defined objects using special methods.

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector2D(self.x + other.x, self.y + other.y)

v1 = Vector2D(1, 2)
v2 = Vector2D(3, 4)
v3 = v1 + v2
print(v3.x, v3.y)
```

4 Composition

Composition is often preferred over inheritance in Python.

```
class Engine:
    def start(self):
        print("Engine started")
```

```
class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
```

5 Functions as First-Class Objects

Functions can be assigned to variables and passed as arguments.

```
def square(x):
    return x * x

f = square
print(f(5))
```

6 Lambda Functions

Lambda functions define small anonymous functions.

```
add = lambda a, b: a + b
print(add(3, 4))
```

7 Map and Filter

```
numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x * x, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers))

print(squares)
print(evens)
```

8 List Comprehensions

List comprehensions provide a concise way to construct lists.

```
squares = [x * x for x in range(5)]
even_squares = [x * x for x in range(10) if x % 2 == 0]
```

9 Common Mistakes and Notes

- The `self` parameter must be explicitly mentioned.
- Python does not enforce access modifiers such as `private` or `protected`.
- Operator overloading should be used sparingly and consistently.
- List comprehensions improve readability when used appropriately.
- Excessive object-oriented design is often unnecessary in Python.

10 Assignments

1. Define a class `Rectangle` with attributes `length` and `breadth`. Add a method to compute the area.
2. Extend the `Rectangle` class to include a method that checks whether the rectangle is a square.
3. Define a class `BankAccount` with attributes `account number` and `balance`. Include methods to deposit and withdraw money.
4. Create a class `Vector2D` that supports addition and subtraction using operator overloading.
5. Write a function that takes a list of integers and returns a list containing their squares using a list comprehension.
6. Given a list of integers, use `map` to compute the cube of each element.
7. Given a list of integers, use `filter` to extract all numbers divisible by 3.
8. Write a function that takes another function and a list as arguments and applies the function to each element of the list.

NUMERICAL COMPUTING WITH NUMPY

1 NumPy Arrays

NumPy provides the ndarray type for numerical computing.

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a)
print(type(a))
```

2 Array Shape and Dimensions

The shape of an array describes its dimensions.

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])

print(b.shape)
print(b.ndim)
```

3 Array Indexing and Slicing

Indexing and slicing operate along dimensions.

```
print(b[0, 1])
print(b[:, 1])
print(b[0, :])
```

4 Reshaping and Transpose

Arrays can be reshaped without changing the data.

```
c = np.arange(12)
d = c.reshape(3, 4)
print(d)

e = d.T
print(e)
```

5 Broadcasting

Broadcasting allows operations on arrays of different shapes.

```
x = np.array([1, 2, 3])
y = np.array([[10],
              [20],
              [30]])

z = x + y
print(z)
```

6 Repeating Rows and Columns

Rows or columns can be repeated explicitly.

```
m = np.array([[1, 2],
               [3, 4]])

row_repeat = np.repeat(m, 2, axis=0)
col_repeat = np.repeat(m, 2, axis=1)

print(row_repeat)
print(col_repeat)
```

7 Basic Statistical Operations

NumPy provides efficient statistical functions.

```
data = np.array([2, 4, 6, 8, 10])

print(np.mean(data))
print(np.median(data))
print(np.std(data))
```

8 Z-Score Normalization

Z-score normalization rescales data to zero mean and unit variance.

```
mean = np.mean(data)
std = np.std(data)

z = (data - mean) / std
print(z)
```

9 Introduction to Matplotlib

Matplotlib is used for basic visualization.

```
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y = x ** 2

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Line Plot")
plt.show()
```

10 Scatter Plot

Scatter plots visualize relationships between variables.

```
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Scatter Plot")
plt.show()
```

11 Common Mistakes and Notes

- NumPy arrays store elements of a single data type.
- Many NumPy operations return views rather than copies.
- Broadcasting avoids explicit loops.
- Axis value 0 refers to rows and 1 refers to columns.
- Explicit loops over array elements are usually unnecessary.

12 Assignments

1. Create a one-dimensional NumPy array containing the first 20 natural numbers. Compute its mean and standard deviation.
2. Create a 3×4 NumPy array using `arange` and `reshape`. Extract the second column using slicing.
3. Given a 2×3 array, repeat each row twice and each column three times.
4. Create two NumPy arrays of shapes $(3, 1)$ and $(1, 4)$. Add them using broadcasting and report the resulting shape.
5. Given a NumPy array of numerical data, perform z-score normalization without using explicit loops.
6. Generate 100 random numbers from a normal distribution and plot their histogram.
7. Create two arrays representing x and y values and produce a scatter plot showing their relationship.
8. Compute the mean of a 4×5 array along rows and along columns using the `axis` argument.

TOPICS INTENTIONALLY NOT COVERED

1. Python virtual environments and package management
2. Advanced input/output operations and file handling
3. Exception handling and custom exceptions
4. Decorators and closures
5. Generators and iterator protocols
6. Context managers (with statement)
7. Type hints and static type checking
8. Advanced object-oriented features (multiple inheritance, abstract base classes)
9. Metaprogramming and reflection
10. Performance optimization and profiling
11. Parallel and concurrent programming
12. Pandas and data frame operations
13. Advanced NumPy features (strides, memory views, masked arrays)
14. Broadcasting rules in full generality
15. Vectorization performance analysis
16. Advanced visualization libraries
17. Scientific computing libraries beyond NumPy
18. Unit testing and debugging tools
19. Packaging and distribution of Python code